

ACH2043

INTRODUÇÃO À TEORIA DA COMPUTAÇÃO

2. Linguagens Livres-do-Contexto:

2.1: Gramáticas Livres-do-Contexto (parte 2)

Referência:

SIPSER, M. Introdução à Teoria da Computação.
2ª edição, Ed. Thomson

Prof. Marcelo S. Lauretto (marcelolauretto@usp.br)

Introdução

- Na aula anterior:
 - Conceitos básicos sobre de Gramáticas Lícres-do-Contexto (GLCs)
- Nesta aula:
 - Estratégias de construção de GLCs
 - Ambiguidade

Projetando gramáticas livres-do-contexto

- Projeto de uma GLC requer certo grau de criatividade
 - Todavia, algumas estratégias podem ser consideradas
 - Exercício: Tente construir GLCs para as linguagens abaixo.
 - Para avaliar cada resposta, simule a derivação de algumas cadeias pela gramática proposta em cada exemplo
 - Nos exemplos abaixo, $\Sigma = \{a, b\}$
1. $L = \{a\}$
 2. $L = \{\varepsilon\}$
 3. $L = \Sigma = \{a, b\}$
 4. $L = \{a\}^*$
 5. $L = \{a\}^+$
 6. $L = \Sigma^* = \{a, b\}^*$
 7. $L = \Sigma^+ = \{a, b\}^+$

Projetando gramáticas livres-do-contexto

8. $L = \{\text{cadeias que começam e terminam com } a\}$

A expressão regular para esta linguagem é: $a\Sigma^*a \cup a$

9. $L = \{\text{cadeias que começam e terminam com o mesmo símbolo}\}$

Note que L pode ser vista como a união $L_1 \cup L_2$, onde

$L_1 = \{\text{cadeias que começam e terminam com } a\}$ e

$L_2 = \{\text{cadeias que começam e terminam com } b\}$.

10. $L = \{a^n b^n \mid n \geq 0\} = \{\varepsilon, ab, aabb, aaabbb, \dots\}$

Note que toda palavra não vazia de L pode ser descrita recursivamente em termos de palavras menores também pertencentes a L

11. $L = \{a^m b^m c^n d^n \mid m, n \geq 0\}$

Note que L pode ser vista como a concatenação $L_1 \circ L_2$, onde

$L_1 = \{a^m b^m \mid m \geq 0\}$ e

$L_2 = \{c^n d^n \mid n \geq 0\}$

Projetando gramáticas livres-do-contexto

- Projeto de uma GLC requer certo grau de criatividade
- Todavia, algumas estratégias podem ser consideradas
- Resoluções dos exemplos dados:
 $\Sigma = \{a, b\}$:

$$1. \quad L = \{a\} \qquad S \rightarrow a$$

$$2. \quad L = \{\varepsilon\} \qquad S \rightarrow \varepsilon$$

$$3. \quad L = \Sigma = \{a, b\} \qquad S \rightarrow a \mid b$$

$$4. \quad L = \{a\}^* \qquad S \rightarrow Sa \mid \varepsilon \qquad \text{ou} \quad S \rightarrow aS \mid \varepsilon$$

$$5. \quad L = \{a\}^+ \qquad S \rightarrow Sa \mid a \qquad \text{ou} \quad S \rightarrow aS \mid a$$

$$6. \quad L = \Sigma^* = \{a, b\}^* \qquad S \rightarrow Sa \mid Sb \mid \varepsilon \qquad \text{ou} \quad S \rightarrow aS \mid bS \mid \varepsilon$$

$$7. \quad L = \Sigma^+ = \{a, b\}^+ \qquad S \rightarrow Sa \mid Sb \mid a \mid b \qquad \text{ou} \quad S \rightarrow aS \mid bS \mid a \mid b$$

Projetando gramáticas livres-do-contexto

8. $L = \{\text{cadeias que começam e terminam com } a\}$

A expressão regular para esta linguagem é: $a\Sigma^*a \cup a$

GLC para $a\Sigma^*a$: “aninhamento” de duas “sub-gramáticas”

$$S \rightarrow aTa$$

$$T \rightarrow Ta \mid Tb \mid \varepsilon$$

GLC final: basta a inclusão da regra $S \rightarrow a$:

$$S \rightarrow aTa \mid a$$

$$T \rightarrow Ta \mid Tb \mid \varepsilon$$

9. $L = \{\text{cadeias que começam e terminam com o mesmo símbolo}\}$

Note que L pode ser vista como a união $L_1 \cup L_2$, onde

$L_1 = \{\text{cadeias que começam e terminam com } a\}$ e

$L_2 = \{\text{cadeias que começam e terminam com } b\}$.

Logo, basta construir uma GLC que reconheça $L_1 \cup L_2$:

$$S \rightarrow S_1 \mid S_2$$

$$S_1 \rightarrow a \mid aTa$$

$$S_2 \rightarrow b \mid bTb$$

$$T \rightarrow Ta \mid Tb \mid \varepsilon$$

Projetando gramáticas livres-do-contexto

$$10. L = \{a^n b^n \mid n \geq 0\} = \{\varepsilon, ab, aabb, aaabbb, \dots\}$$

Note que toda palavra não vazia de L pode ser descrita recursivamente em termos de palavras menores também pertencentes a L :

Denotando por $w_n = a^n b^n$, $n \geq 0$:

- caso base: $w_0 = a^0 b^0 = \varepsilon$
- $w_1 = ab = aw_0 b$
- $w_2 = a^2 b^2 = aabb = aw_1 b$
- $w_3 = a^3 b^3 = aaabbb = aw_2 b$

Ou seja,

$$w_n = \begin{cases} \varepsilon & \text{se } n = 0 \\ a^n b^n = aw_{n-1}b & \text{se } n > 0 \end{cases}$$

Logo, a GLC deve ter uma estrutura recursiva, em que a variável inicial S gere todas as cadeias w_n conforme a equação acima

$$S \rightarrow aSb \mid \varepsilon$$

Projetando gramáticas livres-do-contexto

$$11. L = \{a^m b^m c^n d^n \mid m, n \geq 0\}$$

Note que L pode ser vista como a concatenação $L_1 \circ L_2$, onde

$$L_1 = \{a^m b^m \mid m \geq 0\} \text{ e}$$

$$L_2 = \{c^n d^n \mid n \geq 0\}$$

Já sabemos como construir GLCs para L_1 e para L_2 . O que resta é construir uma GLC que reconheça $L_1 \circ L_2$:

$$\begin{aligned} S &\rightarrow S_1 S_2 \\ S_1 &\rightarrow a S_1 b \mid \varepsilon \\ S_2 &\rightarrow c S_2 d \mid \varepsilon \end{aligned}$$

Projetando gramáticas livres-do-contexto

- Estratégias gerais para construção de GLCs:
 1. Linguagens livres-do-contexto (LLCs) são, usualmente, composições de LLCs mais simples
 - Para projetar uma GLC G para uma linguagem que representa a união entre duas LLCs L_1 e L_2 , construa primeiramente as gramáticas G_1 para L_1 e G_2 para L_2 ; em seguida, combine as regras de G_1 e de G_2 , e adicione uma nova regra
$$S \rightarrow S_1 | S_2 ,$$
onde
 - S , S_1 e S_2 são as variáveis iniciais de G , G_1 e G_2 , respectivamente

Projetando gramáticas livres-do-contexto

1. Linguagens livres-do-contexto (LLCs) são, usualmente, composições de LLCs mais simples (cont)

- Por exemplo, para obter uma gramática para a linguagem $\{0^n 1^n \mid n \geq 0\} \cup \{1^n 0^n \mid n \geq 0\}$, primeiro construímos duas gramáticas mais simples:

$$\begin{array}{ll} \{0^n 1^n \mid n \geq 0\} & S_1 \rightarrow 0S_1 1 \mid \epsilon \\ \{1^n 0^n \mid n \geq 0\} & S_2 \rightarrow 1S_2 0 \mid \epsilon \end{array}$$

Em seguida adicionamos a regra $S \rightarrow S_1 \mid S_2$, resultando na gramática final

$$\begin{array}{l} S \rightarrow S_1 \mid S_2 \\ S_1 \rightarrow 0S_1 1 \mid \epsilon \\ S_2 \rightarrow 1S_2 0 \mid \epsilon \end{array}$$

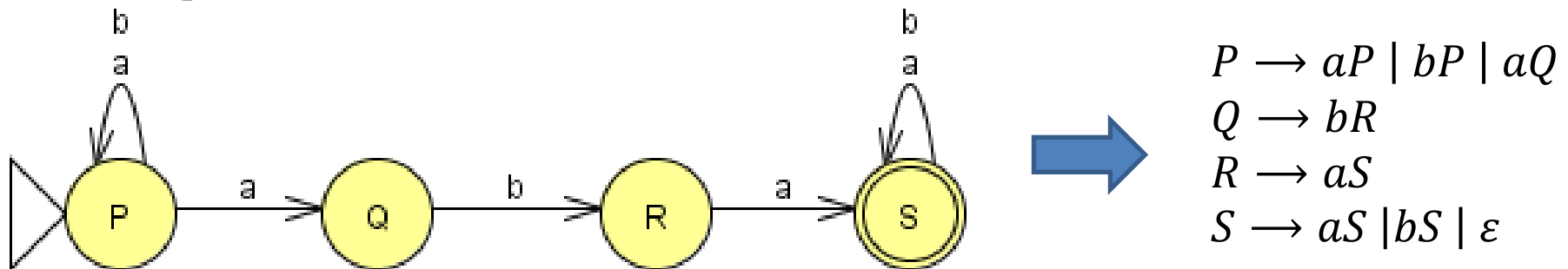
Projetando gramáticas livres-do-contexto

2. Projetar uma GLC para uma linguagem regular é simples, se conseguirmos projetar o AFD, AFN ou expressão regular para ela

– Conversão de um AFD/AFN para uma GLC:

- Crie uma variável R_i para cada estado q_i do autômato
- Para cada transição $q_i \xrightarrow{a} q_j$, adicione a regra $R_i \rightarrow aR_j$;
para cada transição $q_i \xrightarrow{\varepsilon} q_j$, adicione a regra $R_i \rightarrow R_j$
- Para cada estado de aceitação q_i , adicione a regra $R_i \rightarrow \varepsilon$
- Se q_0 é o estado inicial, torne R_0 a variável inicial da gramática

– Exemplo (a): $L = \{w \in \{a, b\}^* \mid w \text{ contém a subcadeia } aba\}$



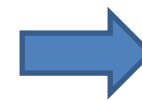
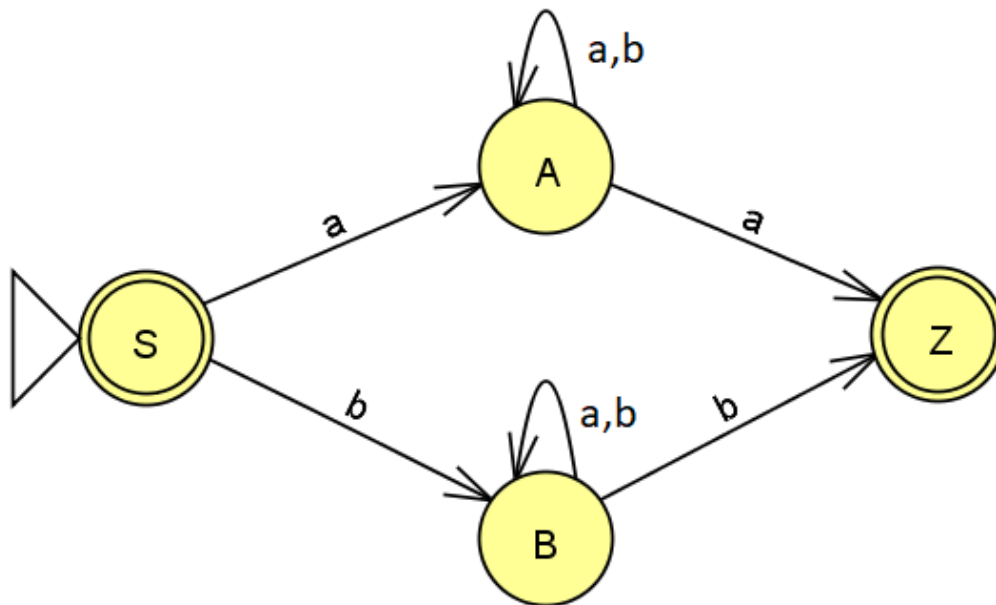
Esta GLC poderia ter uma versão mais compacta (veremos adiante)

Projetando gramáticas livres-do-contexto

2. (cont)

- Exemplo (b):

$L = \{w \in \{a, b\}^* : |w| = 0 \text{ ou } |w| \geq 2 \text{ e começa e termina com o mesmo símbolo} \}$



$$\begin{aligned} S &\rightarrow aA \mid bB \mid \varepsilon \\ A &\rightarrow aA \mid bA \mid aZ \\ B &\rightarrow aB \mid bB \mid bZ \\ Z &\rightarrow \varepsilon \end{aligned}$$

- Exercício: No exemplo 7, apresentamos uma linguagem similar a esta, e mostramos uma estratégia diferente para construir a GLC. Adapte aquela GLC para esta linguagem

Projetando gramáticas livres-do-contexto

2. (cont)

– Conversão de uma expressão regular para uma GLC:

- Se $R = a \rightarrow$ GLC equivalente: composta pela regra $S \rightarrow a$ (idem para ε)
- Se $R = R_1 \cup R_2 \rightarrow$ GLC equivalente: $S \rightarrow S_1 \mid S_2$, onde S_1 e S_2 as variáveis iniciais da GLC equivalente a R_1 e R_2 , respectivamente.
- Se $R = R_1 \circ R_2 \rightarrow$ GLC equivalente: $S \rightarrow S_1 S_2$
- Se $R = R_1^* \rightarrow$ GLC equivalente: $S \rightarrow S_1 S \mid \varepsilon$
- Se $R = R_1^+ \rightarrow$ GLC equivalente: $S \rightarrow S_1 S \mid S_1$

- Exemplo 1: $(a \cup b)^* aba$:

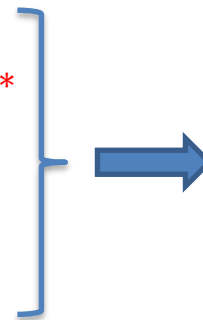
$S \rightarrow T A B A$

$T \rightarrow UT \mid \varepsilon$ representação de $(a \cup b)^*$

$U \rightarrow a \mid b$ representação de $a \cup b$

$A \rightarrow a$

$B \rightarrow b$



Versão mais simples:

$S \rightarrow T aba$

$T \rightarrow aT \mid bT \mid \varepsilon$

Projetando gramáticas livres-do-contexto

2. (cont)

– Conversão de uma expressão regular para uma GLC (cont):

- Exemplo 2: $[a \cup ba^*(a \cup b)a]^*$

$$S \rightarrow TS \mid \varepsilon$$

$$T \rightarrow a \mid bUVa \rightarrow a \cup ba^*(a \cup b)a$$

$$U \rightarrow aU \mid \varepsilon \rightarrow a^*$$

$$V \rightarrow a \mid b \rightarrow a \cup b$$

- Exemplo 3 (apresentado anteriormente, na conversão de AFNs para GLCs):

$$L = \{w \in \{a, b\}^* \mid w \text{ contém a subcadeia } aba\}$$

Expressão regular: $\Sigma^*aba\Sigma^*$

GLC:

$$S \rightarrow TabaT$$

$$T \rightarrow aT \mid bT \mid \varepsilon$$

Projetando gramáticas livres-do-contexto

3. Certas LLCs contêm cadeias com duas subcadeias que são “ligadas” no sentido de que uma máquina para uma linguagem como essa precisaria memorizar uma quantidade de informação ilimitada sobre uma das subcadeias para saber se ela corresponde apropriadamente à outra subcadeia

- Essa situação ocorre na linguagem $\{a^n b^n | n \geq 0\}$, pois uma máquina precisaria memorizar o número de a 's de modo a verificar que ele é igual ao número de b 's
 - Ver Exemplo 10:

$$S \rightarrow aSb \mid \varepsilon$$

- Pode-se construir uma GLC para lidar com essa situação usando uma regra na forma $R \rightarrow uRv$, que gera cadeias nas quais a parte contendo os u 's tem correspondência com a parte contendo os v 's
- Exercício: Adapte a GLC do Exemplo 10 para a linguagem $\{a^n b^{2n} | n \geq 0\}$

Projetando gramáticas livres-do-contexto

4. Em linguagens mais complexas, as cadeias podem conter certas estruturas que aparecem recursivamente como parte de outras estruturas (ou delas mesmas)
- Essa situação ocorre na gramática que gera expressões aritméticas no Exemplo 2.4:
$$\begin{aligned}\langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERMO} \rangle \mid \langle \text{TERMO} \rangle \\ \langle \text{TERMO} \rangle &\rightarrow \langle \text{TERMO} \rangle \times \langle \text{FATOR} \rangle \mid \langle \text{FATOR} \rangle \\ \langle \text{FATOR} \rangle &\rightarrow (\langle \text{EXPR} \rangle) \mid a\end{aligned}$$
 - P.ex., sempre que o símbolo a aparece, uma expressão parentizada inteira pode aparecer recursivamente em seu lugar
 - » Cada operando pode ser uma constante ou uma expressão
 - » Exemplo:
$$(a + a) \times a$$
 - Para atingir esse efeito, coloque o símbolo da variável que gera a estrutura na posição das regras correspondente a onde aquela estrutura pode aparecer recursivamente

Projetando gramáticas livres-do-contexto

- Exercício 2.4:

Give context-free grammars that generate the following languages.
alphabet Σ is $\{0,1\}$.

- ^Aa. $\{w \mid w \text{ contains at least three 1s}\}$
- b. $\{w \mid w \text{ starts and ends with the same symbol}\}$
- c. $\{w \mid \text{the length of } w \text{ is odd}\}$
- ^Ad. $\{w \mid \text{the length of } w \text{ is odd and its middle symbol is a 0}\}$
- e. $\{w \mid w = w^{\mathcal{R}}, \text{ that is, } w \text{ is a palindrome}\}$
- f. The empty set

Ambiguidade

- Às vezes uma gramática pode gerar a mesma cadeia de várias maneiras diferentes
- Tal cadeia terá várias árvores sintáticas diferentes e, portanto, vários significados diferentes
- Se uma gramática gera a mesma cadeia de várias maneiras diferentes, dizemos que a cadeia é **derivada ambigualmente** nessa gramática
- Se uma gramática gera alguma cadeia ambigualmente, dizemos que a gramática é **ambígua**
- Ambiguidade pode ser indesejável em certas aplicações
 - P.ex. em linguagens de programação: um programa deve ter uma única interpretação

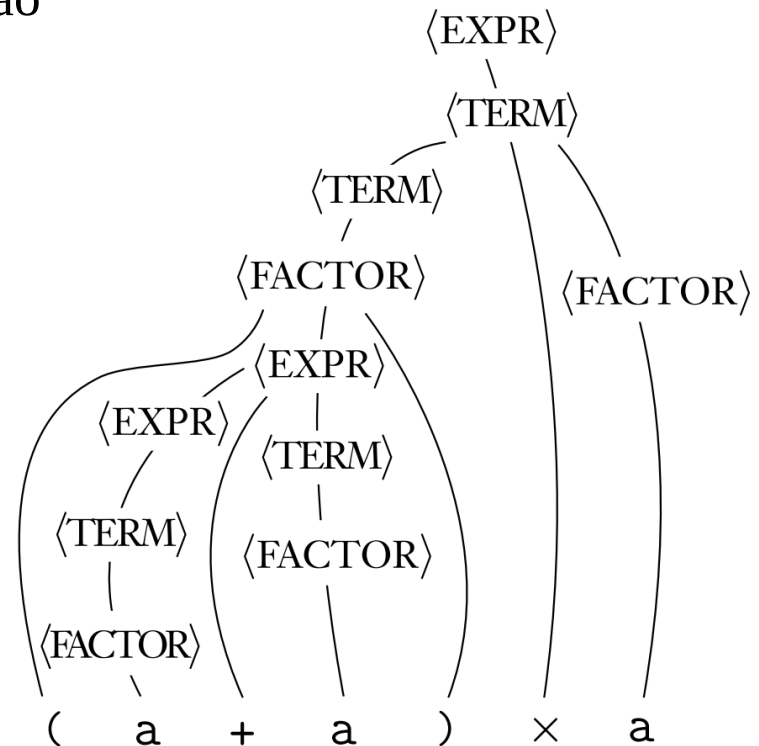
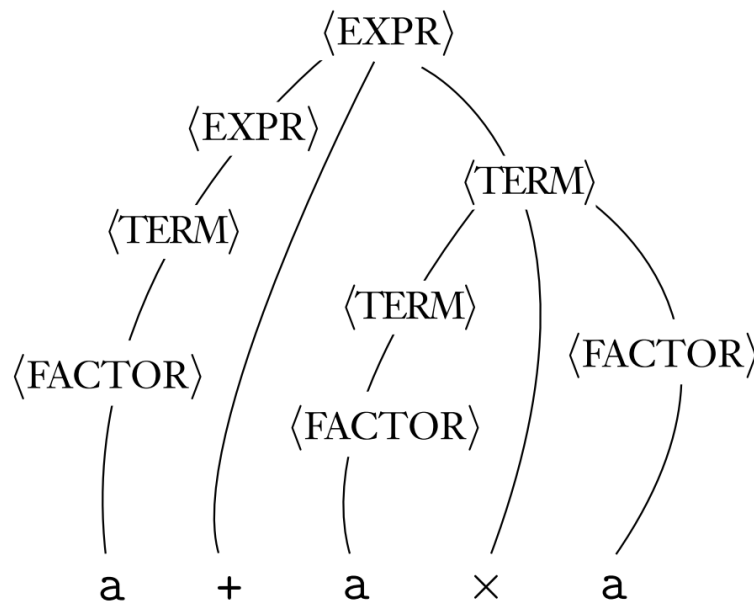
Ambiguidade

- Relembrando o exemplo 2.4 – Gramática G_4 :

$$\begin{aligned}\langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERMO} \rangle \mid \langle \text{TERMO} \rangle \\ \langle \text{TERMO} \rangle &\rightarrow \langle \text{TERMO} \rangle \times \langle \text{FATOR} \rangle \mid \langle \text{FATOR} \rangle \\ \langle \text{FATOR} \rangle &\rightarrow (\langle \text{EXPR} \rangle) \mid a\end{aligned}$$

- Características de G_4 :

- Captura a precedência das operações $\times$ e $+$
- Árvore sintática é única para cada expressão
 $\Rightarrow$ resultado da operação é único



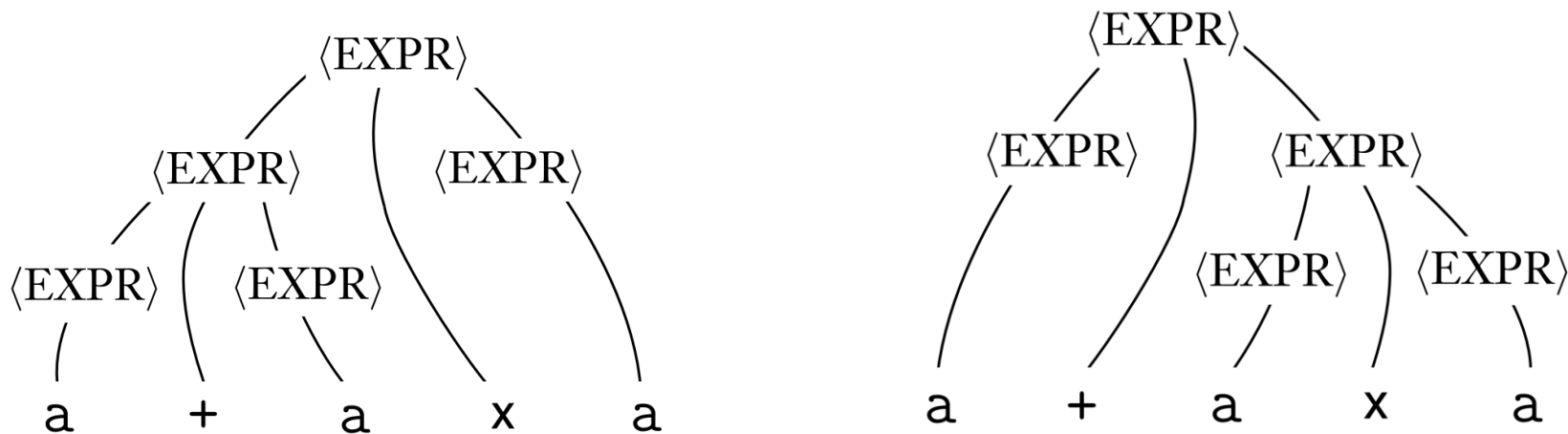
Ambiguidade

- Considere agora a gramática G_5 :

$$\langle \text{EXPR} \rangle \rightarrow \langle \text{EXPR} \rangle + \langle \text{EXPR} \rangle \mid \langle \text{EXPR} \rangle \times \langle \text{EXPR} \rangle \mid (\langle \text{EXPR} \rangle) \mid a$$

- Embora “equivalente” a G_4 (ambas podem gerar as mesmas expressões), G_5 pode gerar cadeias ambigualmente

- Ex: $a + a \times a$



- Qual seria o valor dessa expressão sob cada árvore sintática se $a = 3$?
 - Árvore à esquerda: 18; árvore à direita: 12
- G_4 é não-ambígua, enquanto G_5 é ambígua

Ambiguidade

- A gramática G_2 , que descreve um fragmento da estrutura sintática do Inglês, também é ambígua

$\langle \text{SENTENCE} \rangle \rightarrow \langle \text{NOUN-PHRASE} \rangle \langle \text{VERB-PHRASE} \rangle$
 $\langle \text{NOUN-PHRASE} \rangle \rightarrow \langle \text{CMPLX-NOUN} \rangle \mid \langle \text{CMPLX-NOUN} \rangle \langle \text{PREP-PHRASE} \rangle$
 $\langle \text{VERB-PHRASE} \rangle \rightarrow \langle \text{CMPLX-VERB} \rangle \mid \langle \text{CMPLX-VERB} \rangle \langle \text{PREP-PHRASE} \rangle$
 $\langle \text{PREP-PHRASE} \rangle \rightarrow \langle \text{PREP} \rangle \langle \text{CMPLX-NOUN} \rangle$
 $\langle \text{CMPLX-NOUN} \rangle \rightarrow \langle \text{ARTICLE} \rangle \langle \text{NOUN} \rangle$
 $\langle \text{CMPLX-VERB} \rangle \rightarrow \langle \text{VERB} \rangle \mid \langle \text{VERB} \rangle \langle \text{NOUN-PHRASE} \rangle$
 $\langle \text{ARTICLE} \rangle \rightarrow \text{a} \mid \text{the}$
 $\langle \text{NOUN} \rangle \rightarrow \text{boy} \mid \text{girl} \mid \text{flower}$
 $\langle \text{VERB} \rangle \rightarrow \text{touches} \mid \text{likes} \mid \text{sees}$
 $\langle \text{PREP} \rangle \rightarrow \text{with}$

- Sentença “the girl touches the boy with the flower” tem duas derivações diferentes (e duas interpretações diferentes!)
- Exercício: verifique a afirmação acima
 - Apresente duas árvores sintáticas para a sentença acima

Ambiguidade

- Atenção: quando dizemos que uma gramática gera uma cadeia ambigualmente, queremos dizer que a cadeia tem duas árvores sintáticas diferentes, e não duas derivações diferentes
 - Duas derivações podem diferir meramente pela ordem na qual elas substituem variáveis e ainda assim não na sua estrutura geral
 - Para nos concentrarmos na estrutura da gramática, definimos um tipo de derivação que substitui variáveis em uma ordem fixa

Ambiguidade

- Uma **derivação mais à esquerda** de uma cadeia w é aquela em que, a cada passo, a variável remanescente escolhida para ser substituída é a variável mais à esquerda na forma sentencial intermediária

– Ex: derivação de $(a + a) \times a$ pela gramática G_4 :

$$\begin{aligned}\langle \text{EXPR} \rangle &\rightarrow \langle \text{EXPR} \rangle + \langle \text{TERMO} \rangle \mid \langle \text{TERMO} \rangle \\ \langle \text{TERMO} \rangle &\rightarrow \langle \text{TERMO} \rangle \times \langle \text{FATOR} \rangle \mid \langle \text{FATOR} \rangle \\ \langle \text{FATOR} \rangle &\rightarrow (\langle \text{EXPR} \rangle) \mid a\end{aligned}$$

– Obs: Para uma notação mais compacta, consideramos:

$$\langle E \rangle \equiv \langle \text{EXPR} \rangle, \quad \langle T \rangle \equiv \langle \text{TERMO} \rangle, \quad \langle F \rangle \equiv \langle \text{FATOR} \rangle$$

$$\begin{aligned}\langle E \rangle &\Rightarrow \langle T \rangle \Rightarrow \langle T \rangle \times \langle F \rangle \Rightarrow \langle F \rangle \times \langle F \rangle \Rightarrow (\langle E \rangle) \times \langle F \rangle \\ &\Rightarrow (\langle E \rangle + \langle T \rangle) \times \langle F \rangle \Rightarrow (\langle T \rangle + \langle T \rangle) \times \langle F \rangle \Rightarrow (\langle F \rangle + \langle T \rangle) \times \langle F \rangle \\ &\Rightarrow (a + \langle T \rangle) \times \langle F \rangle \Rightarrow (a + \langle F \rangle) \times \langle F \rangle \Rightarrow (a + a) \times \langle F \rangle \Rightarrow (a + a) \times a\end{aligned}$$

Ambiguidade

- Definição 2.7:

Uma cadeia w é derivada *ambiguamente* na gramática livre-do-contexto G se ela tem duas ou mais derivações mais à esquerda diferentes. A gramática G é *ambígua* se ela gera alguma cadeia ambiguamente.

- Em certas situações, quando temos uma gramática ambígua podemos encontrar uma gramática não-ambígua que gera a mesma linguagem
- Algumas linguagens livres-do-contexto, entretanto, podem ser geradas apenas por gramáticas ambíguas
 - Tais linguagens são chamadas **inerentemente ambíguas**
- Ex: $\{a^i b^j c^k \mid i = j \text{ ou } j = k, i, j, k \geq 0\}$
 - Não há gramática que tenha derivação única para a cadeia aaabbbccc, por exemplo
 - Exercício: apresente uma gramática para a linguagem acima
Note que a linguagem acima pode ser vista como a união de duas linguagens:
 $\{a^i b^i c^* \mid i \geq 0\} \cup \{a^* b^j c^j \mid j \geq 0\}$

Ambiguidade

- Exemplo em linguagens de programação:

- Suponha que uma gramática contenha a regra:

Statement $\rightarrow$ **if** Condition **then** Statement |
 if Condition **then** Statement **else** Statement |

...

Condition $\rightarrow$...

- Algumas estruturas ambíguas podem aparecer:

- A sentença

if a **then** **if** b **then** s1 **else** s2

pode ser interpretada como

if a **then** { **if** b **then** s1 } **else** s2

ou

if a **then** { **if** b **then** s1 **else** s2 }

dependendo da associação do **else** com o 1º ou com o 2º **if**

Ambiguidade

- Exemplo em linguagens de programação:

- Suponha que uma gramática contenha a regra:

Statement $\rightarrow$ **if** Condition **then** Statement |
 if Condition **then** Statement **else** Statement |

...

Condition $\rightarrow$...

- Formas de resolver ambiguidade:

- Modificação da gramática para torná-la não-ambígua

- p.ex tornar **endif** / **fi** obrigatório

Statement $\rightarrow$ **if** Condition **then** Statement **fi** |
 if Condition **then** Statement **else** Statement **fi** |

...

- Manter a gramática ambígua, mas tornar a sentença sensível ao contexto, associando o **else** com o último **if** encontrado

“**if** a **then** **if** b **then** s1 **else** s2”

